
Exploring Clojure

Clojure offers great power through functional style, concurrency support, and clean Java interop. But before you can appreciate all these features, you have to start with the language basics. In this chapter, you will take a quick tour of the Clojure language, including the following:

- Forms
- Reader macros
- Functions
- Bindings and namespaces
- Flow control
- Metadata

If your background is primarily in imperative languages, this tour may seem to be missing key language constructs, such as variables and for loops. [Section 2.7, *Where's My for Loop?*, on page 48](#) will show you how you can *live better* without for loops and variables.

Clojure is very expressive, and this chapter covers many concepts quite quickly. Don't worry if you don't understand every detail; we will revisit these topics in more detail in later chapters. If possible, bring up a REPL, and follow along with the examples as you read.

2.1 Forms

Clojure is *homoiconic*,¹ which is to say that Clojure code is composed of Clojure data. When you run a Clojure program, a part of Clojure called the *reader* reads the text of the program in chunks called *forms* and translates them

1. <http://en.wikipedia.org/wiki/Homoiconicity>

into Clojure data structures. Clojure then compiles and executes the data structures.

The Clojure forms covered in this book are summarized in [Table 1, Clojure forms, on page 23](#). To see forms in action, let's start with some simple forms supporting numeric types.

Using Numeric Types

Numeric literals are forms. Numbers simply evaluate to themselves. If you enter a number, the REPL will give it back to you:

```
42
-> 42
```

A vector of numbers is another kind of form. Create a vector of the numbers 1, 2, and 3:

```
[1 2 3]
-> [1 2 3]
```

A list is also a kind of form. A list is “just data,” but it is also used to call functions. Create a list whose first item names a Clojure function, like the symbol `+`:

```
(+ 1 2)
-> 3
```

As you can see, Clojure evaluates the list as a function call. The style of placing the function first is called *prefix notation*,² as opposed to the more familiar *infix notation* $1 + 2 = 3$. Of course, prefix notation is perfectly familiar for functions whose names are words. For example, most programmers would correctly expect `concat` to come first in this expression:

```
(concat [1 2] [3 4])
-> (1 2 3 4)
```

Clojure is simply being consistent in treating mathematical operators like all other functions and placing them first.

A practical advantage of prefix notation is that you can easily extend it for arbitrary numbers of arguments:

```
(+ 1 2 3)
-> 6
```

2. More specifically, it's called Cambridge Polish notation.

Form	Example(s)	Primary Coverage
Boolean	true, false	Booleans and nil, on page 27
Character	\a	Strings and Characters, on page 25
Keyword	:tag, :doc	Maps, Keywords, and Records, on page 28
List	(1 2 3), (println "foo")	Chapter 3, Unifying Data with Sequences, on page 55
Map	{:name "Bill", :age 42}	Maps, Keywords, and Records, on page 28
Nil	nil	Booleans and nil, on page 27
Number	1, 4.2	Using Numeric Types, on page 22
Set	#{:snap :crackle :pop}	Chapter 3, Unifying Data with Sequences, on page 55
String	"hello"	Strings and Characters, on page 25
Symbol	user/foo, java.lang.String	Symbols, on page 25
Vector	[1 2 3]	Chapter 3, Unifying Data with Sequences, on page 55

Table 1—Clojure forms

Even the degenerate case of no arguments works as you would expect, returning zero. This helps to eliminate fragile, special-case logic for boundary conditions:

```
(+)  
-> 0
```

Many mathematical and comparison operators have the names and semantics that you would expect from other programming languages. Addition, subtraction, multiplication, comparison, and equality all work as you would expect:

```
(- 10 5)  
-> 5
```

```
(* 3 10 10)  
-> 300
```

```
(> 5 2)  
-> true
```

```
(>= 5 5)  
-> true
```

```
(< 5 2)
-> false
```

```
(= 5 2)
-> false
```

Division may surprise you:

```
(/ 22 7)
-> 22/7
```

As you can see, Clojure has a built-in Ratio type:

```
(class (/ 22 7))
-> clojure.lang.Ratio
```

If what you actually want is decimal division, use a floating-point literal for the dividend:

```
(/ 22.0 7)
-> 3.142857142857143
```

If you want to stick to integers, you can get the integer quotient and remainder with quot and rem:

```
(quot 22 7)
-> 3
```

```
(rem 22 7)
-> 1
```

If you need to do arbitrary-precision floating-point math, append M to a number to create a BigDecimal literal:

```
(+ 1 (/ 0.00001 1000000000000000000))
-> 1.0
```

```
(+ 1 (/ 0.00001M 1000000000000000000))
-> 1.00000000000000000000000000001M
```

For arbitrary precision integers, you can append N to create a BigInt literal:

```
(* 1000N 1000 1000 1000 1000 1000 1000)
-> 10000000000000000000000000000N
```

Notice that only one BigInt literal is needed and is contagious to the entire calculation.

Symbols

Forms such as `+`, `concat`, and `java.lang.String` are called *symbols* and are used to name things. For example, `+` names the function that adds things together. Symbols name all sorts of things in Clojure:

- Functions like `str` and `concat`
- “Operators” like `+` and `-`, which are, after all, just functions
- Java classes like `java.lang.String` and `java.util.Random`
- Namespaces like `clojure.core` and Java packages like `java.lang`
- Data structures and references

Symbols cannot start with a number but can consist of alphanumeric characters, plus `+`, `-`, `*`, `/`, `!`, `?`, `.`, and `_`. The list of legal symbol characters is a minimum set that Clojure promises to support. You should stick to these characters in your own code, but do not assume the list is exhaustive. Clojure can use other, undocumented characters in symbols that it employs internally and may add more legal symbol characters in the future. See Clojure’s online documentation³ for updates to the list of legal symbol characters.

Clojure treats `/` and `.` specially in order to support namespaces; see [Namespaces, on page 40](#) for details.

Strings and Characters

Strings are another kind of reader form. Clojure strings are Java strings. They are delimited by double quotes, and they can span multiple lines:

```
"This is a\nmultiline string"
-> "This is a\nmultiline string"

"This is also
a multiline string"
-> "This is also\na multiline string"
```

As you can see, the REPL always shows string literals with escaped newlines. If you actually print a multiline string, it will print on multiple lines:

```
(println "another\nmultiline\nstring")
| another
| multiline
| string
-> nil
```

Clojure does not wrap most of Java’s string functions. Instead, you can call them directly using Clojure’s Java interop forms:

3. <http://clojure.org/reader>

```
(.toUpperCase "hello")
-> "HELLO"
```

The dot before `toUpperCase` tells Clojure to treat it as the name of a Java method instead of a Clojure function.

One string function that Clojure *does* wrap is `toString`. You do not need to call `toString` directly. Instead of calling `toString`, use Clojure's `str` function:

```
(str & args)
```

`str` differs from `toString` in two ways. It smashes together multiple arguments, and it skips `nil` without error:

```
(str 1 2 nil 3)
-> "123"
```

Clojure characters are Java characters. Their literal syntax is `\{letter}`, where `letter` can be a letter or the name of a character: backspace, formfeed, newline, return, space, or tab:

```
(str \h \e \y \space \y \o \u)
-> "hey you"
```

As is the case with strings, Clojure does not wrap Java's character functions. Instead, you can use a Java interop form such as `Character/toUpperCase`:

```
(Character/toUpperCase \s)
-> \S
```

The Java interop forms are covered in [Section 2.5, Calling Java, on page 43](#). For more on Java's `Character` class, see the API documentation at <http://tinyurl.com/java-character>.

Strings are sequences of characters. When you call Clojure sequence functions on a string, you get a sequence of characters back. Imagine that you wanted to conceal a secret message by interleaving it with a second, innocuous message. You could use `interleave` to combine the two messages:

```
(interleave "Attack at midnight" "The purple elephant chortled")
-> (\A \T \t \h \t \e \a \space \c \p \k \u \space \r
 \a \p \t \l \space \e \m \space \i \e \d \l \n \e
 \i \p \g \h \h \a \t \n)
```

That works, but you probably want the resulting sequence as a string for transmission. It is tempting to use `str` to pack the characters back into a string, but that doesn't quite work:

```
(str (interleave "Attack at midnight" "The purple elephant chortled"))
-> "clojure.lang.LazySeq@d4ea9f36"
```

The problem is that `str` works with a variable number of arguments, and you are passing it a *single* argument that contains the argument list. The solution is apply:

```
(apply f args* argseq)
```

`apply` takes a function `f`, some optional `args`, and a sequence of `args` called `argseq`. It then calls `f`, unrolling `args` and `argseq` into an argument list. Use `(apply str ...)` to build a string from a sequence of characters:

```
(apply str (interleave "Attack at midnight" "The purple elephant chortled"))
-> "AThttea cpku raptl em iedlneipghhatn"
```

You can use `(apply str ...)` again to reveal the message:

```
(apply str (take-nth 2 "AThttea cpku raptl em iedlneipghhatn"))
-> "Attack at midnight"
```

The call to `(take-nth 2 ...)` takes every second element of the sequence, extracting the obfuscated message.

Booleans and nil

Clojure's rules for booleans are easy to understand:

- `true` is true, and `false` is false.
- In addition to `false`, `nil` also evaluates to false when used in a boolean context.
- Other than `false` and `nil`, *everything else* evaluates to true in a boolean context.

Lisp programmers be warned: the empty list is not false in Clojure:

```
;           (if part)           (else part)
(if () "We are in Clojure!" "We are in Common Lisp!")
-> "We are in Clojure!"
```

C programmers be warned: zero is not false in Clojure, either:

```
;           (if part)           (else part)
(if 0 "Zero is true" "Zero is false")
-> "Zero is true"
```

A *predicate* is a function that returns either true or false. In Clojure, it is idiomatic to name predicates with a trailing question mark, for example `true?`, `false?`, `nil?`, and `zero?`:

```
(true? expr)
```

```
(false? expr)
```

```
(nil? expr)
```

```
(zero? expr)
```

`true?` tests whether a value is actually true, *not* whether the value evaluates to true in a boolean context. The only thing that is `true?` is `true` itself:

```
(true? true)
-> true
```

```
(true? "foo")
-> false
```

`nil?` and `false?` work the same way. Only `nil` is `nil?`, and only `false` is `false?`.

`zero?` works with any numeric type, returning `true` if it is zero:

```
(zero? 0.0)
-> true
```

```
(zero? (/ 22 7))
-> false
```

There are many more predicates in Clojure. To review them, enter `(find-doc #"\?")` at the REPL.

Maps, Keywords, and Records

A Clojure *map* is a collection of key/value pairs. Maps have a literal form surrounded by curly braces. You can use a map literal to create a lookup table for the inventors of programming languages:

```
(def inventors {"Lisp" "McCarthy" "Clojure" "Hickey"})
-> #'user/inventors
```

The value `"McCarthy"` is associated with the key `"Lisp"`, and the value `"Hickey"` is associated with the key `"Clojure"`.

If you find it easier to read, then you can use commas to delimit each key/value pair. Clojure doesn't care. It treats commas as whitespace:

```
(def inventors {"Lisp" "McCarthy", "Clojure" "Hickey"})
-> #'user/inventors
```

Maps are functions. If you pass a key to a map, it will return that key's value, or it will return `nil` if the key is not found:

```
(inventors "Lisp")
-> "McCarthy"
```

```
(inventors "Foo")
-> nil
```


You can also use the more verbose get function:

```
(get the-map key not-found-val?)
```

get allows you to specify a different return value for missing keys:

```
(get inventors "Lisp" "I dunno!")
-> "McCarthy"
```

```
(get inventors "Foo" "I dunno!")
-> "I dunno!"
```

Because Clojure data structures are immutable and implement hashCode correctly, *any Clojure data structure can be a key in a map*. That said, a very common key type is the Clojure keyword.

A *keyword* is like a symbol, except that keywords begin with a colon (:). Keywords resolve to themselves:

```
:foo
-> :foo
```

This is different from symbols, which want to refer *to* something:

```
foo
-> CompilerException java.lang.RuntimeException:
    Unable to resolve symbol: foo in this context
```

The fact that keywords resolve to themselves makes keywords useful as keys. You could redefine the inventors map using keywords as keys:

```
(def inventors {:Lisp "McCarthy" :Clojure "Hickey"})
-> #'user/inventors
```

Keywords are also functions. They take a map argument and look themselves up in the map. Having switched to keyword keys for the inventors, you can look up an inventor by calling the map or by calling a key:

```
(inventors :Clojure)
-> "Hickey"
```

```
(:Clojure inventors)
-> "Hickey"
```

This flexibility in ordering comes in handy when calling higher-order functions, such as the reference and agent APIs in [Chapter 5, State, on page 113](#).

If several maps have keys in common, you can document (and enforce) this fact by creating a record with defrecord:

```
(defrecord name [arguments])
```

The argument names are converted to keys that have the values passed in when creating the record. Use `defrecord` to create a `Book` record:

```
(defrecord Book [title author])
-> user.Book
```

Then, you can instantiate a record with `user.Book`:

```
(->Book "title" "author")
```

Once you instantiate a `Book`, it behaves almost like any other map:

```
(def b (->Book "Anathem" "Neal Stephenson"))
-> #'user/b

b
-> #:user.Book{:title "Anathem", :author "Neal Stephenson"}

(:title b)
-> "Anathem"
```

Records also have alternative invocations. There is the original syntax that you may have already seen:

```
(Book. "Anathem" "Neal Stephenson")
-> #user.Book{:title "Anathem", :author "Neal Stephenson"}
```

You can also instantiate a record using the literal syntax. This is done by typing in exactly what you have seen returned to you at the REPL. The only difference you will notice is that record literals have to be fully qualified:

```
#user.Book{:title "Infinite Jest", :author "David Foster Wallace"}
-> #user.Book{:title "Infinite Jest", :author "David Foster Wallace"}
```

So far, you have seen numeric literals, lists, vectors, symbols, strings, characters, booleans, records, and `nil`. The remaining forms are covered later in the book, as they are needed. For your reference, see [Table 1, Clojure forms, on page 23](#), which lists all the forms used in the book, a brief example of each, and a pointer to more complete coverage.

2.2 Reader Macros

Clojure forms are read by the *reader*, which converts text into Clojure data structures. In addition to the basic forms, the Clojure reader also recognizes a set of *reader macros*.⁴ Reader macros are special reader behaviors triggered by prefix *macro characters*.

4. Reader macros are totally different from macros, which are discussed in [Chapter 7, Macros, on page 165](#).

The most familiar reader macro is the comment. The macro character that triggers a comment is the semicolon (;), and the special reader behavior is “ignore everything else up to the end of this line.”

Reader macros are abbreviations of longer list forms and are used to reduce clutter. You have already seen one of these. The quote character (') prevents evaluation:

```
'(1 2)
-> (1 2)
```

'(1 2) is equivalent to the longer (quote (1 2)):

```
(quote (1 2))
-> (1 2)
```

The other reader macros are covered later in the book. In the following table, you'll find a quick syntax overview and references to where each reader macro is covered.

Reader Macro	Example(s)	Primary Coverage
Anonymous function	#{.toUpperCase %}	Section 2.3, <i>Functions</i>, on page 32
Comment	; single-line comment	Section 2.2, <i>Reader Macros</i>, on page 30
Deref	@form => (deref form)	Chapter 5, <i>State</i>, on page 113
Metadata	^metadata form	Section 2.8, <i>Metadata</i>, on page 51
Quote	'form => (quote form)	Section 2.1, <i>Forms</i>, on page 21
Regex pattern	#"foo" => a java.util.regex.Pattern	Seq-ing Regular Expressions, on page 72
Syntax-quote	`x	Section 7.3, <i>Making Macros Simpler</i>, on page 172
Unquote	~	Section 7.3, <i>Making Macros Simpler</i>, on page 172
Unquote-splicing	~@	Section 7.3, <i>Making Macros Simpler</i>, on page 172
Var-quote	#'x => (var x)	Chapter 5, <i>State</i>, on page 113

Clojure does not allow programs to define new reader macros. The rationale for this has been explained (and debated) on the Clojure mailing list.⁵ If you

5. <http://tinyurl.com/clojure-reader-macros>

come from a Lisp background, this may be frustrating. We feel your pain. But this compromise in flexibility gives Clojure a more stable core. Custom reader macros could make Clojure programs less interoperable and more difficult to read.

2.3 Functions

In Clojure, a function call is simply a list whose first element resolves to a function. For example, this call to `str` concatenates its arguments to create a string:

```
(str "hello" " " "world")
-> "hello world"
```

Function names are typically hyphenated, as in `clear-agent-errors`. If a function is a predicate, then by convention its name should end with a question mark. As an example, the following predicates test the type of their argument, and all end with a question mark:

```
(string? "hello")
-> true
```

```
(keyword? :hello)
-> true
```

```
(symbol? 'hello)
-> true
```

To define your own functions, use `defn`:

```
(defn name doc-string? attr-map? [params*] body)
```

The `attr-map` associates metadata with the function's var. It's covered separately in [Section 2.8, Metadata, on page 51](#). To demonstrate the other components of a function definition, create a greeting function that takes a name and returns a greeting preceded by "Hello":

```
src/examples/exploring.clj
```

```
(defn greeting
  "Returns a greeting of the form 'Hello, username.'"
  [username]
  (str "Hello, " username))
```

You can call `greeting`:

```
(greeting "world")
-> "Hello, world"
```

You can also consult the documentation for `greeting`:

```
user=> (doc greeting)
-----
exploring/greeting
([username])
  Returns a greeting of the form 'Hello, username.'
```

What does greeting do if the caller omits username?

```
(greeting)
-> ArityException Wrong number of args (0) passed to: user$greeting
    clojure.lang.AFn.throwArity (AFn.java:437)
```

Clojure functions enforce their *arity*, that is, their expected number of arguments. If you call a function with an incorrect number of arguments, Clojure will throw an `ArityException`. If you want to make greeting issue a generic greeting when the caller omits username, you can use this alternate form of `defn`, which takes multiple argument lists and method bodies:

```
(defn name doc-string? attr-map?
  ([params*] body)+)
```

Different arities of the same function can call one another, so you can easily create a zero-argument greeting that delegates to the one-argument greeting, passing in a default username:

```
src/examples/exploring.clj
(defn greeting
  "Returns a greeting of the form 'Hello, username.'
  Default username is 'world'."
  ([] (greeting "world"))
  ([username] (str "Hello, " username))))
```

You can verify that the new greeting works as expected:

```
(greeting)
-> "Hello, world"
```

You can create a function with variable arity by including an ampersand in the parameter list. Clojure will bind the name after the ampersand to a sequence of all the remaining parameters.

The following function allows two people to go on a date with a variable number of chaperones:

```
src/examples/exploring.clj
(defn date [person-1 person-2 & chaperones]
  (println person-1 "and" person-2
    "went out with" (count chaperones) "chaperones.))

(date "Romeo" "Juliet" "Friar Lawrence" "Nurse")
| Romeo and Juliet went out with 2 chaperones.
```

Variable arity is very useful in recursive definitions. See [Chapter 4, *Functional Programming*, on page 85](#) for examples.

Writing function implementations differing by arity is useful. But if you come from an object-oriented background, you'll want *polymorphism*, that is, different implementations that are selected by *type*. Clojure can do this and a whole lot more. See [Chapter 8, *Multimethods*, on page 187](#) and [Chapter 6, *Protocols and Datatypes*, on page 143](#) for details.

`defn` is intended for defining functions at the top level. If you want to create a function from within another function, you should use an anonymous function form instead.

Anonymous Functions

In addition to named functions with `defn`, you can also create anonymous functions with `fn`. There are at least three reasons to create an anonymous function:

- The function is so brief and self-explanatory that giving it a name makes the code harder to read, not easier.
- The function is being used only from inside another function and needs a local name, not a top-level binding.
- The function is created inside another function for the purpose of closing over some data.

Filter functions are often brief and self-explanatory. For example, imagine that you want to create an index for a sequence of words, and you do not care about words shorter than three characters. You can write an `indexable-word?` function like this:

```
src/examples/exploring.clj
(defn indexable-word? [word]
  (> (count word) 2))
```

Then, you can use `indexable-word?` to extract indexable words from a sentence:

```
(require '[clojure.string :as str])
(filter indexable-word? (str/split "A fine day it is" #"\\W+"))
-> ("fine" "day")
```

The call to `split` breaks the sentence into words, and then `filter` calls `indexable-word?` once for each word, returning those for which `indexable-word?` returns true.

Anonymous functions let you do the same thing in a single line. The simplest anonymous `fn` form is the following:

```
(fn [params*] body)
```

With this form, you can plug the implementation of `indexable-word?` directly into the call to `filter`:

```
(filter (fn [w] (> (count w) 2)) (str/split "A fine day" #"\\W+"))
-> ("fine" "day")
```

There is an even shorter syntax for anonymous functions, using implicit parameter names. The parameters are named `%1`, `%2`, and so on. You can also use `%` for the first parameter. This syntax looks like this:

```
 #(body)
```

You can rewrite the call to `filter` with the shorter anonymous form:

```
(filter #(> (count %) 2) (str/split "A fine day it is" #"\\W+"))
-> ("fine" "day")
```

A second motivation for anonymous functions is wanting a named function but only inside the scope of another function. Continuing with the `indexable-word?` example, you could write this:

```
src/examples/exploring.clj
```

```
(defn indexable-words [text]
  (let [indexable-word? (fn [w] (> (count w) 2))]
    (filter indexable-word? (str/split text #"\\W+"))))
```

The `let` binds the name `indexable-word?` to the same anonymous function you wrote earlier, this time inside the lexical scope of `indexable-words`. (`let` is covered in more detail under [Section 2.4, Vars, Bindings, and Namespaces, on page 36](#).) You can verify that `indexable-words` works as expected:

```
(indexable-words "a fine day it is")
-> ("fine" "day")
```

The combination of `let` and an anonymous function says the following to readers of your code: “The function `indexable-word?` is interesting enough to have a name but is relevant only inside `indexable-words`.”

A third reason to use anonymous functions is when you create a function dynamically at runtime. Earlier, you implemented a simple greeting function. Extending this idea, you can create a `make-greeter` function that creates greeting functions. `make-greeter` will take a `greeting-prefix` and return a new function that composes greetings from the `greeting-prefix` and a name.

```
src/examples/exploring.clj
```

```
(defn make-greeter [greeting-prefix]
  (fn [username] (str greeting-prefix " " username)))
```

It makes no sense to name the fn, because it is creating a *different* function each time make-greeter is called. However, you may want to name the results of specific calls to make-greeter. You can use def to name functions created by make-greeter:

```
(def hello-greeting (make-greeter "Hello"))
-> #'user/hello-greeting

(def aloha-greeting (make-greeter "Aloha"))
-> #'user/aloha-greeting
```

Now, you can call these functions, just like any other functions:

```
(hello-greeting "world")
-> "Hello, world"

(aloha-greeting "world")
-> "Aloha, world"
```

Moreover, there is no need to give each greeter a name. You can simply create a greeter and place it in the first (function) slot of a form:

```
((make-greeter "Howdy") "pardner")
-> "Howdy, pardner"
```

As you can see, the different greeter functions remember the value of greeting-prefix at the time they were created. More formally, the greeter functions are *closures* over the value of greeting-prefix.

When to Use Anonymous Functions

Anonymous functions have a terse syntax that is not always appropriate. You may actually prefer to be explicit and create named functions such as `indexable-word?`. That's perfectly fine and will certainly be the right choice if `indexable-word?` needs to be called from more than one place.

Anonymous functions are an option, not a requirement. Use the anonymous forms only when you find that they make your code more readable. They take a little getting used to, so don't be surprised if you gradually use them more and more.

2.4 Vars, Bindings, and Namespaces

When you define an object with `def` or `defn`, that object is stored in a Clojure *var*. For example, the following `def` creates a var named `user/foo`:

```
(def foo 10)
-> #'user/foo
```


The symbol `user/foo` refers to a var that is *bound* to the value 10. If you ask Clojure to evaluate the symbol `foo`, it will return the value of the associated var:

```
foo
-> 10
```

The initial value of a var is called its *root binding*. Sometimes it is useful to have thread-local bindings for a var; this is covered in [Section 5.5, Managing Per-Thread State with Vars, on page 127](#).

You can refer to a var directly. The `var` special form returns a var itself, not the var's value:

```
(var a-symbol)
```

You can use `var` to return the var bound to `user/foo`:

```
(var foo)
-> #'user/foo
```

You will almost never see the `var` form directly in Clojure code. Instead, you will see the equivalent reader macro `#'`, which also returns the var for a symbol:

```
#'foo
-> #'user/foo
```

Why would you want to refer to a var directly? Most of the time, you won't, and you can often simply ignore the distinction between symbols and vars.

But keep in the back of your mind that vars have many abilities other than just storing a value:

- The same var can be aliased into more than one namespace ([Namespaces, on page 40](#)). This allows you to use convenient short names.
- Vars can have metadata ([Section 2.8, Metadata, on page 51](#)). Var metadata includes documentation ([Finding Documentation, on page 18](#)), type hints for optimization, and unit tests.
- Vars can be dynamically rebound on a per-thread basis ([Section 5.5, Managing Per-Thread State with Vars, on page 127](#)).

Bindings

Vars are bound to names, but there are other kinds of bindings as well. For example, in a function call, argument values bind to parameter names. In the following call, 10 binds to the name `number` inside the `triple` function:

```
(defn triple [number] (* 3 number))
-> #'user/triple

(triple 10)
-> 30
```

A function's parameter bindings have a *lexical* scope: they are visible only inside the text of the function body. Functions are not the only way to create a lexical binding. The special form `let` does nothing other than create a set of lexical bindings:

```
(let [bindings*] exprs*)
```

The bindings are then in effect for `exprs`, and the value of the `let` is the value of the last expression in `exprs`.

Imagine that you want coordinates for the four corners of a square, given the bottom, left, and size. You can let the top and right coordinates, based on the values given:

```
src/examples/exploring.clj
(defn square-corners [bottom left size]
  (let [top (+ bottom size)
        right (+ left size)]
    [[bottom left] [top left] [top right] [bottom right]]))
```

The `let` binds `top` and `right`. This saves you the trouble of calculating `top` and `right` more than once. (Both are needed twice to generate the return value.) The `let` then returns its last form, which in this example becomes the return value of `square-corners`.

Destructuring

In many programming languages, you bind a variable to an *entire* collection when you need to access only *part* of the collection.

Imagine that you are working with a database of book authors. You track both first and last names, but some functions need to use only the first name:

```
src/examples/exploring.clj
(defn greet-author-1 [author]
  (println "Hello," (:first-name author)))
```

The `greet-author-1` function works fine:

```
(greet-author-1 {:last-name "Vinge" :first-name "Vernor"})
| Hello, Vernor
```

Having to bind `author` is unsatisfying. You don't need the `author`; all you need is the `first-name`. Clojure solves this with *destructuring*. Any place that you bind

names, you can nest a vector or a map in the binding to reach into a collection and bind only the part you want. Here is a variant of `greet-author` that binds only the first name:

```
src/examples/exploring.clj
(defn greet-author-2 [{fname :first-name}]
  (println "Hello," fname))
```

The binding form `{fname :first-name}` tells Clojure to bind `fname` to the `:first-name` of the function argument. `greet-author-2` behaves just like `greet-author-1`:

```
(greet-author-2 {:last-name "Vinge" :first-name "Vernor"})
| Hello, Vernor
```

Just as you can use a map to destructure any associative collection, you can use a vector to destructure any sequential collection. For example, you could bind only the first two coordinates in a three-dimensional coordinate space:

```
(let [[x y] [1 2 3]]
  [x y])
-> [1 2]
```

The expression `[x y]` destructures the vector `[1 2 3]`, binding `x` to 1 and `y` to 2. Since no symbol lines up with the final element 3, it is not bound to anything.

Sometimes you want to skip elements at the start of a collection. Here's how you could bind only the `z` coordinate:

```
(let [[_ _ z] [1 2 3]]
  z)
-> 3
```

The underscore `()` is a legal symbol and is used idiomatically to indicate “I don't care about this binding.” Binding proceeds from left to right, so the `_` is actually bound twice:

```
; *not* idiomatic!
(let [[_ _ z] [1 2 3]]
  _)
-> 2
```

It is also possible to simultaneously bind both a collection and elements within the collection. Inside a destructuring expression, an `:as` clause gives you a binding for the entire enclosing structure. For example, you could capture the `x` and `y` coordinates individually, plus the entire collection as `coords`, in order to report the total number of dimensions:

```
(let [[x y :as coords] [1 2 3 4 5 6]]
  (str "x: " x ", y: " y ", total dimensions " (count coords)))
-> "x: 1, y: 2, total dimensions 6"
```

Try using destructuring to create an `ellipsize` function. `ellipsize` should take a string and return the first three words followed by

```
src/examples/exploring.clj
```

```
(require '[clojure.string :as str])
(defn ellipsize [words]
  (let [[w1 w2 w3] (str/split words #"\s+")]
    (str/join " " [w1 w2 w3 "..."])))

(ellipsize "The quick brown fox jumps over the lazy dog.")
-> "The quick brown ..."
```

`split` splits the string around whitespace, and then the destructuring form `[w1 w2 w3]` grabs the first three words. The destructuring ignores any extra items, which is exactly what we want. Finally, `join` reassembles the three words, adding the ellipsis at the end.

Destructuring has several other features not shown here and is a mini-language in itself. The Snake game in [Section 5.6, A Clojure Snake, on page 132](#) makes heavy use of destructuring. For a complete list of destructuring options, see the online documentation for `let`.⁶

Namespaces

Root bindings live in a namespace. You can see evidence of this when you start the Clojure REPL and create a binding:

```
user=> (def foo 10)
-> #'user/foo
```

The `user=>` prompt tells you that you are currently working in the `user` namespace.⁷ You should treat `user` as a scratch namespace for exploratory development.

When Clojure resolves the name `foo`, it namespace-qualifies `foo` in the current namespace `user`. You can verify this by calling `resolve`:

```
(resolve sym)
```

`resolve` returns the var or class that a symbol will resolve to in the current namespace. Use `resolve` to explicitly resolve the symbol `foo`:

```
(resolve 'foo)
-> #'user/foo
```

6. http://clojure.org/special_forms

7. Most of the REPL session listings in the book omit the REPL prompt for brevity. In this section, the REPL prompt will be included whenever the current namespace is important.

You can switch namespaces, creating a new one if needed, with `in-ns`:

```
(in-ns name)
```

Try creating a `myapp` namespace:

```
user=> (in-ns 'myapp)
-> #<Namespace myapp>
myapp=>
```

Now you are in the `myapp` namespace, and anything you `def` or `defn` will belong to `myapp`.

When you create a new namespace with `in-ns`, the `java.lang` package is automatically available to you:

```
myapp=> String
-> java.lang.String
```

While you are learning Clojure, you should use the `clojure.core` namespace whenever you move to a new namespace, making Clojure's core functions available in the new namespace as well:

```
myapp=> (clojure.core/use 'clojure.core)
-> nil
```

By default, class names outside `java.lang` must be fully qualified. You cannot just say `File`:

```
myapp=> File/separator
-> java.lang.Exception: No such namespace: File
```

Instead, you must specify the fully qualified `java.io.File`. Note that your file separator character may be different from that shown here:

```
myapp=> java.io.File/separator
-> "/"
```

If you do not want to use a fully qualified class name, you can map one or more class names from a Java package into the current namespace using `import`.

```
(import '(package Class+))
```

Once you import a class, you can use its short name:

```
(import '(java.io InputStream File))
-> java.io.File

(.exists (File. "/tmp"))
-> true
```

import is only for Java classes. If you want to use a Clojure var from another namespace, you must use its fully qualified name or map the name into the current namespace. Take, for example, Clojure's split function that resides in clojure.string:

```
(require 'clojure.string)
(clojure.string/split "Something,separated,by,commas" #"",")
-> ["Something" "separated" "by" "commas"]

(split "Something,separated,by,commas" #"",")
-> Unable to resolve symbol: split in this context
```

To alias split in the current namespace, call require on split's namespace and give it the alias str:

```
(require '[clojure.string :as str])
(str/split "Something,separated,by,commas" #"",")
-> ["Something" "separated" "by" "commas"]
```

The simple form of require shown earlier causes the current namespace to reference *all* public vars in clojure.string and provide access to them under the alias str. This can be confusing, because it does not make explicit which names are being referred to.

It is idiomatic to import Java classes and require namespaces at the top of a source file, using the ns macro:

```
(ns name & references)
```

The ns macro sets the current namespace (available as *ns*) to name, creating the namespace if necessary. The references can include :import, :require, and :use, which work like the similarly named functions to set up the namespace mappings in a single form at the top of a source file. For example, this call to ns appears at the top of the sample code for this chapter:

```
src/examples/exploring.clj
(ns examples.exploring
  (:require [clojure.string :as str])
  (:import (java.io File)))
```

Clojure's namespace functions can do quite a bit more than I have shown here.

You can reflectively traverse namespaces and add or remove mappings at any time. To find out more, issue this command at the REPL. Since we have moved around a bit in the REPL, we will also ensure that we are in the user namespace so that our REPL utilities are available to us:

```
(in-ns 'user)
(find-doc "ns-")
```

Alternately, browse the documentation at <http://clojure.org/namespaces>.

2.5 Calling Java

Clojure provides simple, direct syntax for calling Java code: creating objects, invoking methods, and accessing static methods and fields. In addition, Clojure provides syntactic sugar that makes calling Java from Clojure more concise than calling Java from Java!

Not all types in Java are created equal: the primitives and arrays work differently. Where Java has special cases, Clojure gives you direct access to these as well. Finally, Clojure provides a set of convenience functions for common tasks that would be unwieldy in Java.

Accessing Constructors, Methods, and Fields

The first step in many Java interop scenarios is creating a Java object. Clojure provides the new special form for this purpose:

```
(new classname)
```

Try creating a new Random:

```
(new java.util.Random)
-> <Random java.util.Random@667cbde6>
```

The REPL simply prints out the new Random instance by calling its `toString()` method. To use a Random instance, you will need to save it away somewhere. For now, simply use `def` to save the Random into a Clojure Var:

```
(def rnd (new java.util.Random))
-> #'user/rnd
```

Now you can call methods on `rnd` using Clojure's dot (`.`) special form:

```
(. classname member-symbol & args)
(. classname (member-symbol & args))
```

The `.` can call methods. For example, the following code calls the no-argument version of `nextInt()`:

```
(. rnd nextInt)
-> -791474443
```

Random also has a `nextInt()` that takes an argument. You can call that version by simply adding the argument to the list:

```
(. rnd nextInt 10)
-> 8
```

In the previous call, the `.` form is used to access an instance method. But `.` works with all kinds of class members: fields as well as methods, and statics as well as instances. Here you can see the `.` used to get the value of `pi`:

```
(. Math PI)
-> 3.141592653589793
```

Notice that `Math` is not fully qualified. It doesn't have to be, because Clojure imports `java.lang` automatically. To avoid typing `java.util.Random` everywhere, you could explicitly import it:

```
(import [& import-lists])
; import-list => (package-symbol & class-name-symbols)
```

`import` takes a variable number of lists, with the first part of each list being a package name and the rest being names to import from that package. The following import allows unqualified access to `Random`, `Locale`, and `MessageFormat`:

```
(import '(java.util Random Locale)
        '(java.text MessageFormat))
-> java.text.MessageFormat
```

```
Random
-> java.util.Random
```

```
Locale
-> java.util.Locale
```

```
MessageFormat
-> java.text.MessageFormat
```

At this point, you have almost everything you need to call Java from Clojure. You can do the following:

- Import class names
- Create instances
- Access fields
- Invoke methods

However, there isn't anything particularly exciting about the syntax. It is just "Java with different parentheses."

Javadoc

Although reaching into Java from Clojure is easy, remembering how all of the Java bits underneath work can be daunting. Clojure provides a `javadoc` function

that will make your life much easier. This provides a pleasant experience from REPL when exploring.

```
(javadoc java.net.URL)
->
```

2.6 Flow Control

Clojure has very few flow control forms. In this section, you will meet `if`, `do`, and `loop/recur`. As it turns out, this is almost all you will ever need.

Branch with `if`

Clojure's `if` evaluates its first argument. If the argument is logically true, it returns the result of evaluating its second argument:

```
src/examples/exploring.clj
(defn is-small? [number]
  (if (< number 100) "yes"))

(is-small? 50)
-> "yes"
```

If the first argument to `if` is logically false, it returns `nil`:

```
(is-small? 50000)
-> nil
```

If you want to define a result for the “else” part of `if`, add it as a third argument:

```
src/examples/exploring.clj
(defn is-small? [number]
  (if (< number 100) "yes" "no"))

(is-small? 50000)
-> "no"
```

The `when` and `when-not` control flow macros are built on top of `if` and are described in [when and when-not, on page 171](#).

Introduce Side Effects with `do`

Clojure's `if` allows only one form for each branch. What if you want to do more than one thing on a branch? For example, you might want to log that a certain branch was chosen. `do` takes any number of forms, evaluates them all, and returns the last.

You can use a `do` to print a logging statement from within an `if`:

```
src/examples/exploring.clj
(defn is-small? [number]
  (if (< number 100)
    "yes"
    (do
      (println "Saw a big number" number)
      "no"))))
```

which results in:

```
(is-small? 200)
| Saw a big number 200
-> "no"
```

This is an example of a *side effect*. The `println` doesn't contribute to the return value of `is-small?` at all. Instead, it reaches out into the world outside the function and actually *does something*.

Many programming languages mix pure functions and side effects in a completely ad hoc fashion. Not Clojure. In Clojure, side effects are explicit and unusual. `do` is one way to say “side effects to follow.” Since `do` ignores the return values of all its forms save the last, those forms must have side effects to be of any use at all.

Recur with loop/recur

The Swiss Army knife of flow control in Clojure is `loop`:

```
(loop [bindings *] exprs*)
```

The `loop` special form works like `let`, establishing bindings and then evaluating `exprs`. The difference is that `loop` sets a recursion point, which can then be targeted by the `recur` special form:

```
(recur exprs*)
```

`recur` binds new values for `loop`'s bindings and returns control to the top of the loop. For example, the following `loop/recur` returns a countdown:

```
src/examples/exploring.clj
(loop [result [] x 5]
  (if (zero? x)
    result
    (recur (conj result x) (dec x))))
```

```
-> [5 4 3 2 1]
```

The first time through, `loop` binds `result` to an empty vector and binds `x` to 5. Since `x` is not zero, `recur` then rebinds the names `x` and `result`:

- result binds to the previous result conjoined with the previous x.
- x binds to the decrement of the previous x.

Control then returns to the top of the loop. Since x is again not zero, the loop continues, accumulating the result and decrementing x. Eventually, x reaches zero, and the if terminates the recurrence, returning result.

Instead of using a loop, you can recur back to the top of a function. This makes it simple to write a function whose entire body acts as an implicit loop:

```
src/examples/exploring.clj
(defn countdown [result x]
  (if (zero? x)
      result
      (recur (conj result x) (dec x))))

(countdown [] 5)
-> [5 4 3 2 1]
```

recur is a powerful building block. But you may not use it very often, because many common recursions are provided by Clojure's sequence library.

For example, countdown could also be expressed as any of these:

```
(into [] (take 5 (iterate dec 5)))
-> [5 4 3 2 1]

(into [] (drop-last (reverse (range 6))))
-> [5 4 3 2 1]

(vec (reverse (rest (range 6))))
-> [5 4 3 2 1]
```

Do not expect these forms to make sense yet—just be aware that there are often alternatives to using recur directly. The sequence library functions used here are described in [Section 3.2, Using the Sequence Library, on page 60](#). Clojure *will not* perform automatic tail-call optimization (TCO). However, it will optimize calls to recur. [Chapter 4, Functional Programming, on page 85](#) defines TCO and explores recursion and TCO in detail.

At this point, you have seen quite a few language features but still no variables. Some things really do vary, and [Chapter 5, State, on page 113](#) will show you how Clojure deals with changeable *references*. But most variables in traditional languages are unnecessary and downright dangerous. Let's see how Clojure gets rid of them.

2.7 Where's My for Loop?

Clojure has no for loop and no direct mutable variables.⁸ So, how do you write all that code you are accustomed to writing with for loops?

Rather than create a hypothetical example, we decided to grab a piece of open source Java code (sort of) randomly, find a method with some for loops and variables, and port it to Clojure. We opened the Apache Commons project, which is very widely used. We selected the StringUtils class in Commons Lang, assuming that such a class would require little domain knowledge to understand. We then browsed for a method that had multiple for loops and local variables and found `indexOfAny`:

```
data/snippets/StringUtils.java
// From Apache Commons Lang, http://commons.apache.org/lang/
public static int indexOfAny(String str, char[] searchChars) {
    if (isEmpty(str) || ArrayUtils.isEmpty(searchChars)) {
        return -1;
    }
    for (int i = 0; i < str.length(); i++) {
        char ch = str.charAt(i);
        for (int j = 0; j < searchChars.length; j++) {
            if (searchChars[j] == ch) {
                return i;
            }
        }
    }
    return -1;
}
```

`indexOfAny` walks `str` and reports the index of the first char that matches any char in `searchChars`, returning -1 if no match is found.

Here are some example results from the documentation for `indexOfAny`:

```
StringUtils.indexOfAny(null, *)           = -1
StringUtils.indexOfAny("", *)             = -1
StringUtils.indexOfAny(*, null)           = -1
StringUtils.indexOfAny(*, [])             = -1
StringUtils.indexOfAny("zzabyycdxx", ['z', 'a']) = 0
StringUtils.indexOfAny("zzabyycdxx", ['b', 'y']) = 3
StringUtils.indexOfAny("aba", ['z'])      = -1
```

8. Clojure provides *indirect* mutable references, but these must be explicitly called out in your code. See [Chapter 5, State, on page 113](#) for details.

There are two ifs, two fors, three possible points of return, and three mutable local variables in `indexOfAny`, and the method is fourteen lines long, as counted by David A. Wheeler's SLOCCount.⁹

Now let's build a Clojure index-of-any, step by step. If we just wanted to find the matches, we could use a Clojure filter. But we want to find the *index* of a match. So, we create `indexed`, a function that takes a collection and returns an indexed collection:

```
src/examples/exploring.clj
(defn indexed [coll] (map-indexed vector coll))
```

`indexed` returns a sequence of pairs of the form `[idx elt]`. Try indexing a string:

```
(indexed "abcde")
-> ([0 \a] [1 \b] [2 \c] [3 \d] [4 \e])
```

Next, we want to find the indices of all the characters in the string that match the search set.

Create an index-filter function that is similar to Clojure's `filter` but that returns the indices instead of the matches themselves:

```
src/examples/exploring.clj
(defn index-filter [pred coll]
  (when pred
    (for [[idx elt] (indexed coll) :when (pred elt)] idx)))
```

Clojure's `for` is *not* a loop but a sequence comprehension (see [Transforming Sequences, on page 66](#)). The index/element pairs of `(indexed coll)` are bound to the names `idx` and `elt` but only when `(pred elt)` is true. Finally, the comprehension yields the value of `idx` for each matching pair.

Clojure sets are functions that test membership in the set. So, you can pass a set of characters and a string to `index-filter` and get back the indices of all characters in the string that belong to the set. Try it with a few different strings and character sets:

```
(index-filter #{\a \b} "abcdbbb")
-> (0 1 4 5 6)

(index-filter #{\a \b} "xyz")
-> ()
```

At this point, we have accomplished *more* than the stated objective. `index-filter` returns the indices of all the matches, and we need only the first index. So, `index-of-any` simply takes the first result from `index-filter`:

9. <http://www.dwheeler.com/sloccount/>

```
src/examples/exploring.clj
```

```
(defn index-of-any [pred coll]
  (first (index-filter pred coll)))
```

Test that `index-of-any` works correctly with a few different inputs:

```
(index-of-any #{\z \a} "zzabyycdxx")
-> 0
(index-of-any #{\b \y} "zzabyycdxx")
-> 3
```

The Clojure version is simpler than the imperative version by every metric (see [Table 2, *Relative complexity of imperative and functional indexOfAny*, on page 51](#)). What accounts for the difference?

- The imperative `indexOfAny` must deal with several special cases: null or empty strings, a null or empty set of search characters, and the absence of a match. These special cases add branches and exits to the method. With a functional approach, most of these kinds of special cases just work without any explicit code.
- The imperative `indexOfAny` introduces local variables to traverse collections (both the string and the character set). By using higher-order functions such as `map` and sequence comprehensions such as `for`, the functional `index-of-any` avoids all need for variables.

Unnecessary complexity tends to snowball. For example, the special case branches in the imperative `indexOfAny` use the magic number `-1` to indicate a nonmatch. Should the magic number be a symbolic constant? Whatever you think the right answer is, *the question itself disappears* in the functional version. While shorter and simpler, the functional `index-of-any` is also *vastly more general*:

- `indexOfAny` searches a string, while `index-of-any` can search any sequence.
- `indexOfAny` matches against a set of characters, while `index-of-any` can match against any predicate.
- `indexOfAny` returns the first match, while `index-filter` returns all the matches and can be further composed with other filters.

As an example of how much more general the functional `index-of-any` is, you could use code like we just wrote to find the third occurrence of “heads” in a series of coin flips:

```
(nth (index-filter #{:h} [:t :t :h :t :h :t :t :t :h :h])
  2)
-> 8
```

Metric	LOC	Branches	Exits/Method	Variables
Imperative version	14	4	3	3
Functional version	6	1	1	0

Table 2—Relative complexity of imperative and functional `indexOfAny`

So, it turns out that writing `index-of-any` in a functional style, without loops or variables, is simpler, less error prone, and more general than the imperative `indexOfAny`.¹⁰ On larger units of code, these advantages become even more telling.

2.8 Metadata

The Wikipedia entry on metadata¹¹ begins by saying that metadata is “data about data.” That is true but not usably specific. In Clojure, metadata is data that is *orthogonal to the logical value of an object*. For example, a person’s first and last names are plain old data. The fact that a person object can be serialized to XML has nothing to do with the person and is metadata. Likewise, the fact that a person object is dirty and needs to be flushed to the database is metadata.

Reader Metadata

The Clojure language itself uses metadata in several places. For example, vars have a metadata map containing documentation, type information, and source information. Here is the metadata for the `str` var:

```
(meta #'str)
-> {:ns #<Namespace clojure.core>,
   :name str,
   :file "core.clj",
   :line 313,
   :arglists ([] [x] [x & ys]),
   :tag java.lang.String,
   :doc "With no args, ... etc."}
```

Some common metadata keys and their uses are shown in [Table 3, Common metadata keys, on page 52](#).

10. It is worth mentioning that you could write a functional `indexOfAny` in plain Java, although it would not be idiomatic. It may become more idiomatic when closures are added to the language. See <http://functionaljava.org/> for more information.

11. <http://en.wikipedia.org/wiki/Metadata>

Metadata Key	Used For
:arglists	Parameter info used by doc
:doc	Documentation used by doc
:file	Source file
:line	Source line number
:macro	True for macros
:name	Local name
:ns	Namespace
:tag	Expected argument or return type

Table 3—Common metadata keys

Much of the metadata on a var is added automatically by the Clojure compiler. To add your own key/value pairs to a var, use the metadata reader macro:

```
^metadata form
```

For example, you could create a simple shout function that upcases a string and then document that shout both expects and returns a string, using the :tag key:

```
; see also shorter form below
(defn ^{:tag String} shout [{:tag String} s] (.toUpperCase s))
-> #'user/shout
```

You can inspect shout's metadata to see that Clojure added the :tag:

```
(meta #'shout)
-> {:arglists ([s]),
    :ns #<Namespace user>,
    :name shout,
    :line 32,
    :file "NO_SOURCE_FILE",
    :tag java.lang.String}
```

You provided the :tag, and Clojure provided the other keys. The :file value NO_SOURCE_FILE indicates that the code was entered at the REPL.

Because :tag metadata is so common, you can also use the short-form ^Class-name, which expands to ^{:tag Classname}. Using the shorter form, you can rewrite shout as follows:

```
(defn ^String shout [^String s] (.toUpperCase s))
-> #'user/shout
```


If you find the metadata disruptive when you are reading the definition of a function, you can place the metadata last. Use a variant of `defn` that wraps one or more body forms in parentheses, followed by a metadata map:

```
(defn shout
  ([s] (.toUpperCase s))
  {:tag String})
```

2.9 Wrapping Up

This has been a long chapter. But think about how much ground you have covered: you can instantiate basic literal types, define and call functions, manage namespaces, and read and write metadata. You can write purely functional code, and yet you can easily introduce side effects when you need to do so. You have also met Lisp concepts including reader macros, special forms, and destructuring.

The material here would take hundreds of pages to cover in most other languages. Is the Clojure way really that much simpler? Yes, in part. Half the credit for this chapter belongs to Clojure. Clojure's elegant design and abstraction choices make the language much easier to learn than most.

That said, the language may not *seem* so easy to learn right now. That's because we are taking advantage of Clojure's power to move much faster than most programming language books.

So, the other half of the credit for this chapter belongs to you, the reader. Clojure will give back what you put in, and then some. Take the time you need to feel comfortable with the chapter's examples and with using the REPL. The rest of the book will give you the opportunity to do that.